Worksheet 1: Event Loop & Async Behaviour

1.Define Event Loop

The event loop is the core mechanism in Node.js that continuously checks for work to do and decides when callbacks, timers and promise handlers should run, one after another on the main thread.
It lets Node.js handle many asynchronous tasks (I/O, timers, promises) without blocking the single JavaScript thread.

2.Why is Node.js called single-threaded but non-blocking?

- Node.js runs your JavaScript code on a single main thread so, only one piece of JS executes at a time (single-threaded).
- Expensive operations like file/network I/O are offloaded to the OS or a thread pool and their results come back via callbacks so, the main thread is free to keep processing other work (non-blocking).

3.Difference between Call Stack and Callback Queue.

Call Stack: The place where currently running functions are stored it grows when functions are called and shrinks when they return.

Callback Queue (task queue): A waiting line of callbacks from timers, I/O, etc. When the call stack is empty the event loop takes the next callback from this queue and pushes it to the stack to execute.

4.Practical Task

Start
End
Promise
Timeout

Explain WHY

- `console.log("Start")` runs immediately as normal synchronous code so ,Start prints first.

- `console.log("End")` is still synchronous so, it runs next and prints "End" while the current turn of the event loop is still running.

- `Promise.resolve().then(...)` schedules a microtask (promise job) to run after the current synchronous code finishes but before macrotasks.

- `setTimeout(..., 0)` schedules a macrotask (timer callback) for later it does not run immediately.

Worksheet 2: Async Patterns (Callback → Promise → Async/Await)

Convert this callback code into Promise-based code.

promises.js > ...

```javascript
const fs = require("fs");

function readFilePromise(path) {
  return new Promise((resolve, reject) => {
    fs.readFile(path, (err, data) => {
      if (err) reject(err);
      else resolve(data);
    });
  });
}

readFilePromise("data.txt")
  .then(data => {
    console.log(data.toString());
  })
  .catch(err => {
    console.log(err);
  });
```

Rewrite the Promise version using async/await.

```
async > JS await.js > ...
1   async function readData() {
2     try {
3       const data = await readFilePromise("data.txt");
4       console.log(data.toString());
5     } catch (err) {
6       console.log(err);
7     }
8   }
9
10  readData();
11
```

- Add proper error handling using try/catch.

  Put await calls inside a try block.

  Handle failures in the catch block (log, return error response, etc.).

Worksheet 3: Error Handling in Node.js

1. Difference between sync error and async error.

   Synchronous error: Happens immediately while the code is running on the call stack for example a thrown error in normal function code or invalid JSON parsing. You can catch it directly with a surrounding try/catch because it happens in the same turn of execution.

   Asynchronous error: Happens later inside a callback, promise or async/await operation (like fs.readFile, database calls, timers). You must handle it in the callback, with .catch, or with try/catch around await, because it occurs after the original function has finished.

2.What is unhandledRejection?

unhandledRejection is a special process event that fires when a Promise is rejected and there is no .catch or rejection handler anywhere in its chain. In modern Node.js, such unhandled rejections can log a warning and may terminate the process with a non-zero exit code, so you should either handle them or listen to process.on("unhandledRejection") for last-resort logging.

3.What error occurs?

Because missing.txt does not exist, fs.readFile rejects its Promise with a file-system error of type ENOENT ("no such file or directory"). The error object includes properties like code: 'ENOENT', errno, syscall: 'open', and path: 'missing.txt'.

4.How to safely handle it?

```
import fs from "fs/promises";


async function readFileSafe() {
  try {
    const data = await fs.readFile("missing.txt", "utf8");
    console.log(data);
  } catch (err) {
    if (err.code === "ENOENT") {
      console.error("File not found, creating a default one...");
      // handle missing file: create it, return default data, etc.
    } else {
      console.error("Unexpected error while reading file:", err);
    }
  }
}
```

readFileSafe();

1. Why are events useful in large applications?

   They decouple parts of the system: one module emits an event, and many other modules can react without tight coupling or direct function calls.
   They improve scalability and responsiveness, because many actions can be triggered asynchronously when something happens (user login, HTTP request finished, job completed) without blocking the main flow.

   2.Give one real-world example.

   A chat or messaging app backend is a classic example: when a user sends a message, the server emits an event (like message:new), and different listeners log it, store it in the database, and push it to connected users in real time.

1. What problem do streams solve?

Streams solve the problem of handling large or continuous data without loading everything into memory at once. They let you process data piece by piece as it arrives, which is more efficient for memory and faster to start sending/processing results (good for big files, video, logs, etc.).

2. Difference between readFile() and createReadStream()

- fs.readFile()
  - Reads the entire file into memory before you use it.
  - Simple and good for small files, but can be slow or memory-heavy for large files.
- fs.createReadStream()
  - Returns a readable stream and reads the file in chunks (default ~64 KB) instead of all at once.
  - Much better for large files and allows piping directly to another stream (like an HTTP response or write stream) with low memory usage.

3.

```javascript
import fs from "fs";

// create a readable stream from a file
const readable = fs.createReadStream("bigfile.txt", {
  encoding: "utf8",
});

// option 1: manually handle chunks
readable.on("data", (chunk) => {
  console.log("Chunk received:", chunk.length);
});

readable.on("end", () => {
  console.log("Finished reading file");
});

readable.on("error", (err) => {
  console.error("Error while reading:", err);
});

// option 2: pipe to another stream (e.g., copy file)
const writable = fs.createWriteStream("copy.txt");
readable.pipe(writable);
```

1.What happens with a very large file?

- Out of memory crash — Node.js has a memory limit (around 1.4–2 GB by default). Files exceeding this will cause an ENOMEM or heap allocation error and crash the process.
- Blocking other operations — While that huge buffer is being built in memory, your app's performance degrades because the garbage collector is under pressure and other requests are starved of resources.
- Hard size cap — readFile() cannot even handle files larger than 2 GiB due to an internal buffer limit, regardless of how much RAM you have

2.Why is this better than readFile()?

createReadStream() reads the file in small chunks (default ~64 KB each) and emits them one at a time via the data event, so only a tiny portion of the file is in memory at any moment.

Task 1: /about route

You add another `else if` block that checks `req.url === "/about"` and `req.method === "GET"`, then send back a response with `res.end()`. This is basic routing — matching the URL path and HTTP method to decide what to respond with.

Task 2: JSON response

To return JSON, you need to do two things: set the `Content-Type` header to `application/json` using `res.writeHead()`, and convert a JavaScript object to a string with `JSON.stringify()` before passing it to `res.end()`.

```js
const data = { message: "Hello from API", status: "success" };

res.writeHead(200, { "Content-Type": "application/json" });

res.end(JSON.stringify(data));
```

Without `JSON.stringify()`, the client would receive `[object Object]` instead of actual JSON.

Task 3: Proper status codes

Use `res.writeHead(statusCode, headers)` to set the HTTP status code before sending the response. Common status codes to use:

- 200 — OK (successful GET request)
- 201 — Created (successful POST, e.g., new resource created)
- 404 — Not Found (unknown route or missing resource)
- 500 — Internal Server Error